

1 Part A

1.1 Learning Aim

My learning aim is to understand better the limitations, benefits, and detriments of a reinforcement learning algorithm used in robot control.

Reinforcement learning has the ability to improve performance beyond that of a classic algorithm since it has knowledge of a reward signal and is designed to exploit that signal for maximum reward.

An issue with existing robot controllers is that they may be suboptimal, may not handle dynamic environments well, or may not be sufficiently intelligent to complete complex tasks. Neural nets trained via reinforcement learning can handle all three of these deficiencies, if properly set up and trained.

The aim of the learning algorithm is to learn a policy to control a differential drive robot to navigate toward a goal, given a dataset of experiences which it collects during exploration. To simplify the problem, the goal will always be the same cell.

1.2 Learning Algorithm

I'll be using deep Q-Learning (and specifically SARSA) as my reinforcement learning (RL) algorithm. One reason is that I use RL in my PhD research and this will give me an opportunity to understand the algorithm more deeply. As part of this, I wrote my own neural net and RL library from scratch.

Deep Q-Learning works by defining a critic, the Q network, which evaluates (s, a) inputs and outputs q-values, where s is the state vector and a is the action vector. Q-Learning uses dynamic programming (aka bootstrapping) to learn q-values via the Bellman equation.

Deep Q-Learning suffers from the curse of dimensionality, meaning that it takes exponentially more data to fully map the search space as the dimension of the search space increases. With this in mind, I've taken care to reduce the size of the map and the number of actions to a reasonable amount. Details in subsequent sections.

The actor will be the Epsilon-Greedy algorithm, which chooses a random action with probability equal to ϵ , and chooses the best action with probability $1 - \epsilon$. Details in subsequent sections.

I expect Deep Q-Learning to be able to learn the problem because historically, Deep Q-Learning has exhibited great success in learning to do tasks with discrete action spaces, and play board games like Chess and Go. This task is not too dissimilar from these.

1.3 Problem Framing

The inputs to the algorithm will be the state, (x, y, θ) , and the action, $(a_{[0,1,2]})$, and the output is a single q-value. Action primitives are used in place of continuous actions to simplify the learning problem. Three action primitives are available; turn left, turn right, and go straight. These map to motion-model commands of $(v = 0, \omega = +\pi/2)$, $(v = 0, \omega = -\pi/2)$, $(v = cellwidth, \omega = 0.0)$ respectively. The cell-width is set to 2.0m to reduce the map to 24 grid cells. A simple linear (with respect to the inputs) motion model with $dt = 1.0$ is used so that at each step, the robot fully achieves its control, (v, w) . Again, this is done to simplify the learning problem.

There will be a sparse reward of 10.0 if the goal was reached and 0.0 otherwise. In addition, there's a movement penalty is -0.01 if the robot moves to an empty cell, and -1 if the cell is occupied. Lastly, there's a penalty of -1.0 if the robot leaves the map and goes out of bounds.

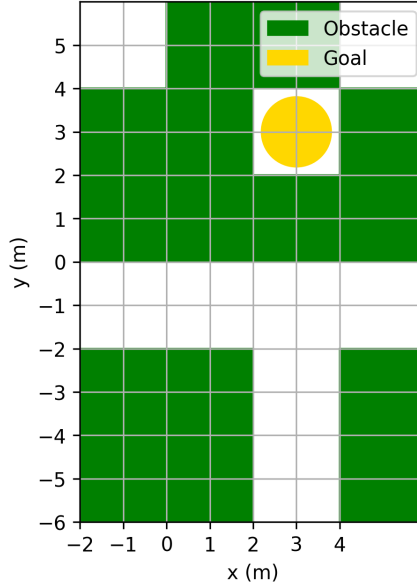
An episode is ended when either a timeout occurs, or the robot goes out of bounds, or the robot reaches the goal.

The neural net will be parameterized by a number of learnable parameters which is a function of the network size (more details later), and their values will be tuned via stochastic gradient descent, and back-propagation. Each dense layer has $m \cdot n + m$ learnable parameters, where n is the input vector length, and m is the number of neurons in the layer. Each activation has no learnable parameters. Details in later sections.

The map and obstacles from DS1 will be used to create an environment. The motion model used in previous assignments will be used to drive the robot through the environment. Over the course of many episodes, the robot will explore online and collect data to create a replay buffer (a.k.a. a dataset).

fig. 1 shows a map of the environment, complete with obstacles and a static goal.

Figure 1: Map of the environment. Cell-Width was 2.0, so the obstacles are inflated by 2x.



Neural Net Derivation

Two types of layers were used: a linear layer and a nonlinear layer (also called an activation). A standard dense layer was used for the linear layer, while leakyRELU was used as the activation. LeakyRELU is a piecewise-linear function where $y = s_- \cdot x$ when $x < 0$ and $y = s_+ \cdot x$ when $x \geq 0$ where s_- and s_+ are scalars and they must not be equal to each other. The activation has no weights. Commonly, s_+ is set to 1.0 and s_- is set to 0.1.

The dense layer followed the standard formulation of $y = Ax + B$ where A and B are the weights matrices. Each layer is the combination of a dense layer and an activation.

The neural net was assembled by feeding the output of the previous layer into the input of the next layer. If $y_i = f_i(x_i)$ is the function for layer i , then $x_i = y_{i-1}$ for all $i > 0$ and $i < n$, where $n = \text{len}(\text{layers})$. x_0 is then the input to the neural net, and y_n is the prediction.

With the knowledge that we'll need both the partial derivatives of each layer with respect to the weights, as well as the inputs, we derive that here.

For a dense layer, $\frac{\partial y}{\partial x} = A$, $\frac{\partial f}{\partial A} = x$, and $\frac{\partial f}{\partial B} = 1$.

For the leakyRELU, $\frac{\partial y}{\partial x} = s_-$ if $x < 0$ and $\frac{\partial y}{\partial x} = s_+$ if $x \geq 0$

Loss Equation

Machine learning is usually framed as an optimization problem, where some defined loss is minimized. Q-Learning is considered a regression problem, so the mean-squared-error is used. This is often written as $\mathcal{L} = \frac{1}{2}(y - y_{g.t.})^2 / \text{len}(\text{batch})$, where $y_{g.t.}$ is the ground truth target and y is the prediction.

In Q-Learning, "boot-strapping", a.k.a. dynamic programming is used to train the model by leveraging its own information about future states. The Bellman equation is used to define the relationship between current and future model outputs. Often, the model is denoted as $Q(s, a)$, and its outputs are called q-values.

$$Q(s, a) = r(s, a) + \gamma \cdot Q(s', a') \quad (1)$$

In eq. (1) we have a choice of where to get a' from. If a' is taken from the dataset, then this is considered off-policy learning. If a' is produced from the actor (in our case, Eps-Greedy), then this is considered on-policy learning [2]. Because I've chosen to use the policy to generate a' , this flavor of Q-Learning is actually the SARSA

algorithm. On-policy algorithms tend to converge faster because the critic is always considering the current policy, instead of some arbitrary other policy.

The goal of the learning algorithm is to change the weight values of Q such that the Bellman equation is true for any state-action pair in our problem. If we re-arrange the terms, we get $0 = Q(s, a) - (r(s, a) + \gamma \cdot Q(s', a'))$. For arbitrary weight values, this equation isn't guaranteed to be zero, so we replace the 0 with an error symbol. $\epsilon = Q(s, a) - (r(s, a) + \gamma \cdot Q(s', a'))$. If we square both sides and multiply by half, we get $\frac{1}{2}\epsilon^2 = \frac{1}{2}(Q(s, a) - (r(s, a) + \gamma \cdot Q(s', a')))^2$. If we set our loss to $\mathcal{L} = \frac{1}{2}\epsilon^2$ then we can see that our prediction is $y = Q(s, a)$ and our ground truth is $y_{g.t.} = r(s, a) + \gamma \cdot Q(s', a')$. From inspection we can see that minimizing our loss will also drive ϵ to zero, so the optimization problem is now well-defined.

We can also define the gradient of the loss with respect to y here: $\frac{\partial \mathcal{L}}{\partial y} = (y_{g.t.} - y)/len(batch)$ which will be used later.

Optimization Derivation

We must now derive a way to solve our optimization problem.

You may recall from calculus that the minimum of a function occurs when the derivative is zero. If we want to minimize $\mathcal{L}$, then we must find its derivative (called the gradient in higher dimensions). For simple systems for which a closed-form solution may be derived, we would set the gradient equal to zero and solve for the input values which achieves that gradient.

Generally, for neural nets a closed-form solution is not available, so instead numerical optimization is used to find some local minima. Our numerical optimization will take the form of gradient descent, where gradients are computed and followed iteratively until some convergence is met. In practice, datasets are often too large to compute the gradient over, so instead the gradient is computed over some random batch of datapoints. This is called stochastic gradient descent. The algorithm is below.

```

Done  $\leftarrow$  False
while not Done do
    Batch  $\leftarrow$  Dataset
     $s_B, a_B \leftarrow$  Batch
     $\mathcal{L}_B \leftarrow f(s_B, a_B, Q)$  for a loss function  $f$ 
     $\nabla_{w_B} \mathcal{L}_B \leftarrow \frac{\partial \mathcal{L}}{\partial w}$  for all  $w$ 
     $w^{t+1} = w^t + \alpha \cdot \nabla_{w_B} \mathcal{L}_B$ 
    Done  $\leftarrow$  isdone()
end while

```

Where t is the step (a.k.a. iteration) count.

Training can be *Done* when either convergence is reached (e.g. the loss hasn't changed more than some value for a number of iterations) or after a max number of training steps. We use a max number of episodes to rollout before setting *Done* to true.

One fundamental question remains ... how do we actually compute $\frac{\partial \mathcal{L}}{\partial w}$?? We can turn to our old friend multi-variable calculus and the chain rule to find it. Recall that the chain rule states that if $f(x) = g(h(x))$ then $\frac{\partial f}{\partial x} = \frac{\partial g}{\partial h} \cdot \frac{\partial h}{\partial x}$. We want to find $\frac{\partial \mathcal{L}}{\partial w}$. Recall that we always feed the output of the previous layer into the input of the subsequent layer. Then $\mathcal{L} = f(y_n(x_n(y_{n-1}(\dots(x_0))))))$.

So for an arbitrary layer i , the gradient $\frac{\partial \mathcal{L}}{\partial w} = \frac{\partial f}{\partial y_n} \cdot \frac{\partial y_n}{\partial x_n} \cdot \dots \cdot \frac{\partial y_i}{\partial w_i}$. We can compute the gradient backwards from the final output all the way to the first input, saving the gradient for each layer's weight along the way. This is called *backpropagation*. This is where the gradients for the loss, the dense layer, and the leakyRELU layer are used.

In algorithm form it looks like this. Keep in mind that a layer output is always defined as y_i and the input is always defined as x_i , and $y_i = f_i(x_i)$. Also, recall that $x_{i+1} = y_i$.

```

Forward Pass:
 $x_0 \leftarrow concatenate(s, a)$ 
for  $layer_i$  in layers do

```

```

     $y_i \leftarrow f_i(x_i)$ 
end for
 $\mathcal{L} = f(y_n)$ 
 $\frac{\partial \mathcal{L}}{\partial y_n} = (y_{g.t.} - y_n) / \text{len}(\text{batch})$ 

```

Backward Pass:

```

for layer in reversed(layers) do
     $\frac{\partial y_i}{\partial x_i} \leftarrow \nabla_{x_i} f_i$ 
     $\frac{\partial y_i}{\partial w_i} \leftarrow \nabla_{w_i} f_i$ 
     $\frac{\partial \mathcal{L}}{\partial y_i} \leftarrow \frac{\partial \mathcal{L}}{\partial x_{i+1}} \cdot \frac{\partial y_i}{\partial x_i}$ 
     $\frac{\partial \mathcal{L}}{\partial w_i} \leftarrow \frac{\partial \mathcal{L}}{\partial x_{i+1}} \cdot \frac{\partial y_i}{\partial w_i}$ 
end for

```

All the gradients of $\mathcal{L}$ with respect to w_i are then used in the stochastic gradient descent algorithm from above.

Learning Tricks

The exploration rate attenuates during training. It starts at 33% and linearly reduces to 2.5% when there's only 15% of the episodes left, and then stays constant: $\alpha = \min(\text{step} / (0.85 \cdot \text{max_steps}), 1.0)$, $\epsilon = 0.33 \cdot (1.0 - \alpha) + 0.025 \cdot \alpha$

Weight Decay is also used. This is the practice of reducing the magnitude of weight values each update step so that the neural net layer outputs don't grow too large. It aids in learning stability and reduces overfitting to the dataset, which would also reduce generalizability. The equation for it is $w' = (1.0 - \beta) \cdot w$, where $\beta = 1e - 6$.

Exploding / Vanishing Gradient

Due to the fact that downstream gradients are multiplied by upstream gradients in the backpropagation algorithm, this means that if the magnitude of individual layer's gradients is larger than 1, the product of gradients will grow to infinity. Similarly, if the magnitude of individual layer's gradients is less than 1, the product of gradients will shrink to zero.

Two common practices to combat this are by normalizing each layer's inputs before passing them through the layer, or by using residual layers. A residual layer simply means the output of the layer adds the input onto it so that gradients can backpropagate through the summation to the layer's input unchanged. This trick only works when the input and output of a layer are the same size. Neither of these techniques are used in my code, but they are easy enough to implement if needed.

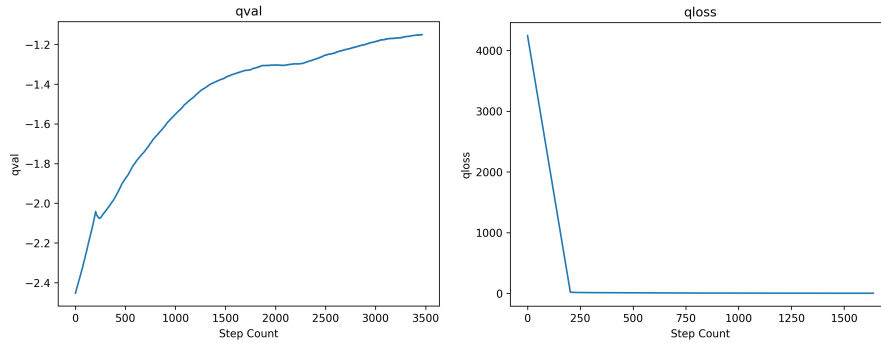
1.4 Code

Much of the detail of the learning algorithm are contained in other sections. I'll briefly state that running the algorithm sequentially on one data point at a time was extremely slow, and that by using a batch instead, the algorithm became more computationally efficient.

1.5 Run on Example Set

I started directly on my *DS1* map, and experimented with different learning rates, batch sizes, and network architectures. These experiments are not reported as they were primarily used for debugging. The example set shows that the learning algorithm is correctly working as intended (fig. 2).

Figure 2: Some metrics for my example set. These plots show that learning is occurring, that losses are dropping, and that q-values are converging.



2 Part B

2.1 Parameters

Some parameters were kept constant for all reported training. They are as follows.

Discount, $\gamma = 0.95$. Discount is used in the Bellman equation to determine how much influence future actions have. A value of 0.0 means only consider the immediate reward, while a value of 1.0 means weight future rewards the same as current rewards. Usually, values between 0.9 and 0.99 are used. The lower the value, the easier the training of the critic tends to be.

Batch size, $B = 256$. Batch size determines how many samples to use to compute the stochastic gradient. The larger the batch size, the more consistent the gradient will be, as well as the closer it will be to the true gradient of the dataset. However, larger batches require more memory and are slower to compute. Smaller batch sizes result in less required memory and compute but less stable training.

Episode timeout, $max_len_ep = 100$. The max an episode can go until it's stopped and a penalty is attributed. Episodes don't often timeout, they more often end due to out-of-bounds penalty or a goal-reached reward.

Max exploration, $\epsilon = 0.333$. The exploration rate determines how often random actions are taken, instead of optimal actions (according to the critic). At the beginning of training you want high exploration, and at the end of training (and during evaluation) you want low exploration.

State length, $len(s) = 3$. The state was (x, y, θ) .

Number of action primitives, $nb_actions = 3$. There was only one action input, and it could take on values in $[0, 1, 2]$, corresponding to [left turn, right turn, straight ahead]. By limiting the number of action primitives, the search space is greatly reduced, making the learning problem easier, and requiring less data to converge.

Other constants are brought up in the appropriate sections.

2.2 Evaluation Measures

Metrics such as success rate, time-to-goal, and energy usage were recorded. Metrics to track how well learning is happening were also recorded, including average q-values, average episodic cumulative reward, and critic loss.

fig. 3 shows the results of the best performing policy. The two primary metrics, success rate and time-to-goal, were about 95% and about 5 steps, respectively. Other metrics such as episodic reward, q-loss, and q-val show convergence by the end of training. Lastly, the exploration rate, ϵ , is plotted during training, showing that it started at 33% and linearly reduced to 2.5% after 85% of episodes before staying constant.

Figure 3: Best Results. L.R. 1e-2, Network Size 32 neurons, 5 layers, Episode count: 30,000. Number of parameters: 4321

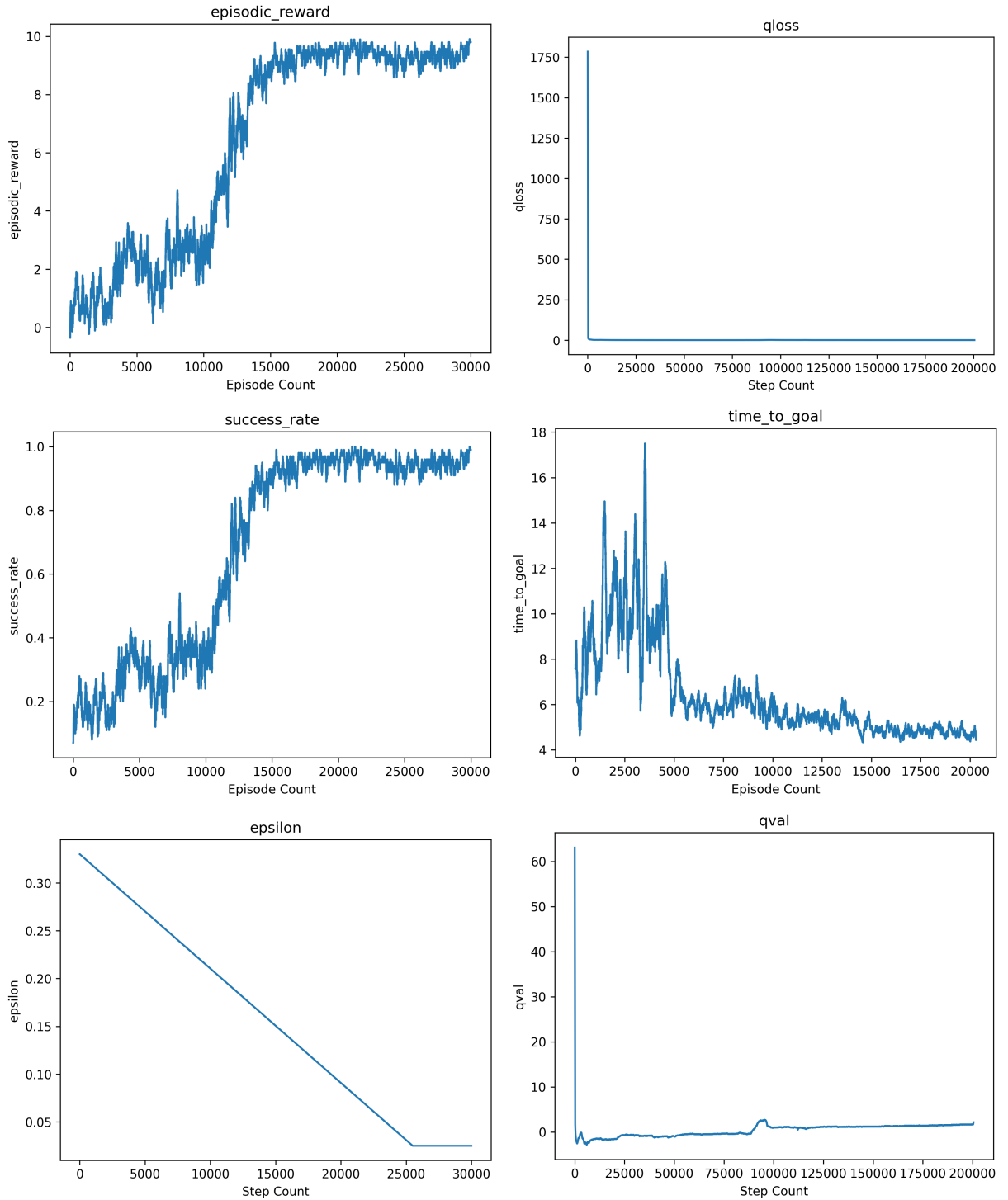
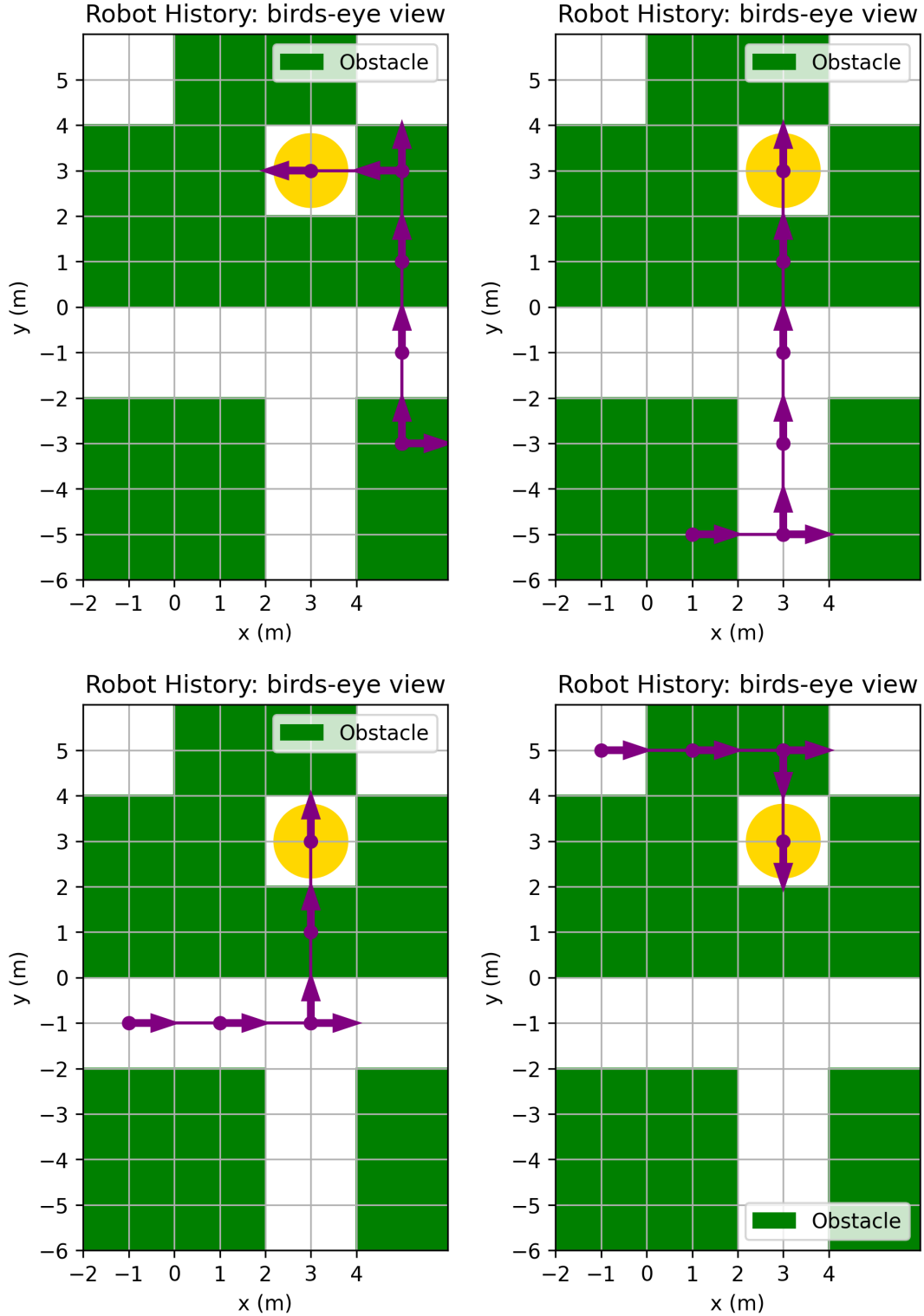


fig. 4 shows example rollouts from the beta policy at the end of training for several interesting start locations. Clearly, the policy favors traversing through non-obstacle regions, which would make sense because the epsilon-greedy action technique is set to all greedy at the end of training, meaning that the action with the highest q-value (denoted a^*) will always be chosen (eq. (2)). Since open spaces have lower "energy" (aka movement) costs, the Q-network has learned that actions which take the robot into open spaces will result in higher cumulative reward.

$$a^* = \operatorname{argmax}_a (Q(s, a)) \forall a \quad (2)$$

Figure 4: Best policy example rollouts. Obstacles in green, goal in gold, robot path in purple. Notice that in situations where feasible paths exist by going through obstacles or through open space, the policy has learned to use the open space because it will accrue a higher cumulative reward.



2.3 Learning Parameters Study

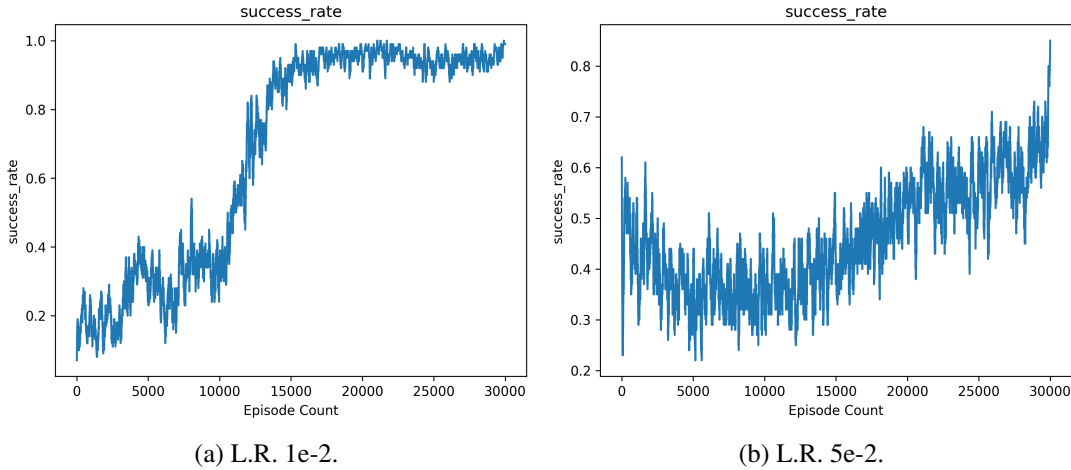
The effect of learning rate on convergence was tested by running training with three learning rates and measuring time to convergence.

If learning rate is too high, it can cause large changes in parameter values, leading to unpredictable policy behavior, as well as potential algorithm non-convergence.

From my tests, when learning rate was $1e-1$, it resulted in exploding parameter values, eventually causing a stack overflow (hence why there's only two plots in fig. 5). When learning rate was larger (but not large enough to cause stack overflows, value $5e-2$), the max success rate was less than when learning rate is lower (l.r. $1e-2$): 60% vs 95% success rate (fig. 5). This is because the optimization algorithm converges to a higher loss if the gradient descent step size is larger. Recall $w^{t+1} = w^t + \alpha \cdot \nabla_{w_B} \mathcal{L}_B$. If α (the learning rate) is bigger, then the values of w_t will have a larger upper bound (assuming $\mathcal{L}_B$ is not zero. A loss of zero only happens if the policy has overfit the data).

One solution to learning rate sensitivity is to use "momentum", which is basically just a low pass filter on the gradient values. Think of the loss function like a hilly landscape. We want to settle on the bottom of a valley, but if learning rate is too high we might bounce around from one side of the valley to another, forever. Momentum dampens this bouncing by averaging gradient values over multiple time-steps, thereby making it easier to reach the local minima with various learning rates [1]. Momentum isn't used in this project, but would be fairly easy to implement.

Figure 5: Learning Rate Comparison. Network size of 32 neurons, 5 layers used for each. 30,000 episodes rolled out. Number of parameters: 4321.



Network Size

The size of the network was varied and the success rate was recorded. There are two main dials we have access to. The first is number of layers, and the second is number of neurons per dense layer.

Recall that a dense layer's output is $y = Ax + B$, where x is a column vector of size $n \times 1$ of the previous layer's outputs, A is $m \times n$ and B is a column vector of size $m \times 1$. From any class on linear algebra, we know that m corresponds to the number of equations in the system and that each entry in the $m \times 1$ column vector, Ax , is a weighted sum of the input x .

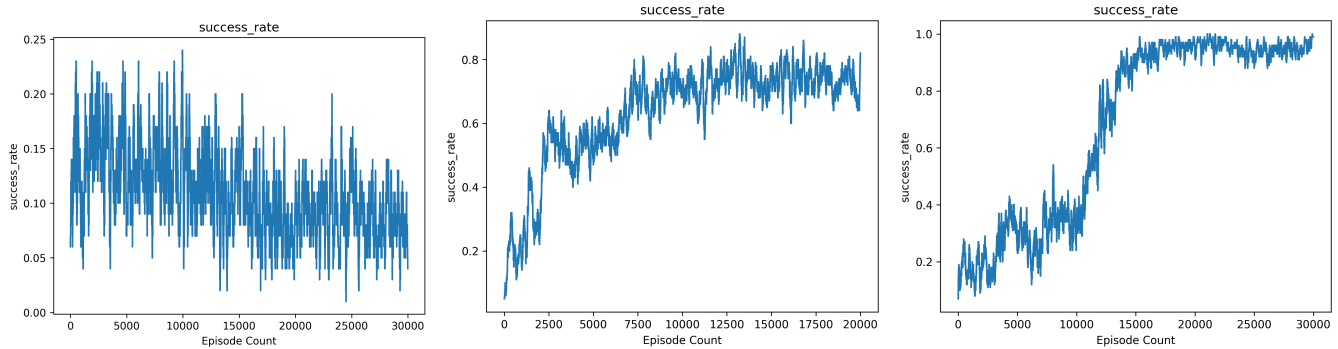
As we increase the number of neurons per dense layer, m , we increase the number of different lines that the dense layer produces.

The downside of increasing network size is that with more weights to learn, the more data we need, so larger networks tend to require more data and more time to converge.

fig. 6 shows the success rate as a function of network size. With the smallest network (33 parameters), the success rate never surpasses 25%. With the medium network (2209 parameters), the success rate converged on about 80%.

The largest network (4321 parameters) converges to about 95% success rate.

Figure 6: Network Size Comparison. L.R. $1e-2$ used for each. 30,000 episodes rolled out for (a) and (c). 20,000 episodes rolled out for (b). The difference in number of episodes is due to how long it took to converge. (b) was quicker to converge, so fewer episodes were needed.



(a) Network size: 4 neurons, 1 layer. (b) Network size: 32 neurons, 3 layers. (c) Network size: 32 neurons, 5 layers.
Number of parameters: 33 Number of parameters: 2209 Number of parameters: 4321

It makes sense that a more complex model with more learnable parameters would achieve a higher success rate because more neurons and more layers means more linear functions, along with more non-linear activations, respectively. More linear functions means more different linear combinations of the inputs, and more non-linear activations means arbitrary non-linear functions can be more accurately modeled.

2.4 Algorithm Analysis

In summary, the Q-Learning algorithm exhibited good behavior under certain conditions. In no particular order, if the learning rate was small enough so that the minimum loss was small, but not too small such that training took exceptionally long to converge. If the network had enough parameters to learn the complexity of the problem, without too many parameters such that much more data and compute would be needed to converge. If the learning problem was simple enough. As a rule of thumb, this is the case when the number of data-points available is at least a magnitude larger than the number of parameters in the model. Math in prior sections.

In contrast, the Q-Learning algorithm behaved poorly when the following occurred. The learning rate was so big that weight values diverged and caused a stack overflow. The network was so small that little of the non-linear complexity of the problem could be learned. Math in prior sections.

More poor behavior: if the dimension of the search space is so large that it suffered from the "curse of dimensionality". That is, in search algorithms like A* or Q-Learning, if the search space is partitioned into cells of constant width, with n cells in one dimension, then the number of distinct states is n^d , where d is the number of dimensions. When searching is 2d, this isn't terrible as it's just n^2 . But in more practical robotics problems, d might be anywhere between 10 and 100. Obviously n^{100} is such an incredibly large number that a search algorithm won't be able to visit every state in the space. So Q-Learning's ability to predict q-values from an arbitrary state-action pair will be hindered as the ratio of data to number of distinct states grows smaller. This results in either poor performance on the control problem if too little data is used, or exceptionally large compute, data, and time needed to converge.

References

- [1] Diederik P. Kingma and Jimmy Ba. Adam: A method for stochastic optimization, 2017.
- [2] G. Rummery and Mahesan Niranjan. On-line q-learning using connectionist systems. *Technical Report CUED/F-INFENG/TR 166*, 11 1994.